

SOFTWARE PROJECT DOCUMENTATION REPORT

Project Title: CareerConnect – A Modern Job
Board Application

PREPARED BY

Mangesh Parbhane

Independently Developed Project

Date: June 14, 2026

Version: 1.0 (Frontend Release)

Table of Contents

1. Executive Summary
2. Introduction
3. Problem Statement
4. Proposed Solution
5. System Architecture
6. Technology Stack
7. Functional Features
8. User Experience (UX) Design Decisions
9. Development Process
10. GitHub Repository Management
11. CI/CD Pipeline
12. Deployment
13. Testing and Validation
14. Challenges Encountered and Solutions
15. Future Enhancements
16. Conclusion
17. References
18. Appendix

3. Executive Summary

The modern employment landscape relies heavily on digital platforms to connect job seekers with potential employers. CareerConnect is a comprehensive, responsive, and intuitive Job Board Web Application designed to streamline this critical process. This project, independently developed by Mangesh Parbhane, focuses heavily on delivering a premium frontend user experience utilizing cutting-edge web technologies such as React, TypeScript, Vite, and Tailwind CSS.

The primary goal of CareerConnect is to provide a seamless interface where candidates can discover opportunities and recruiters can manage applications efficiently. The frontend prototype includes essential features such as advanced job searching, role-based dashboards (Candidate and Recruiter), and a highly responsive design that caters to both desktop and mobile users. Through the strategic use of Artificial Intelligence in the development workflow, the application was built rapidly while adhering to industry best practices in coding, CI/CD pipeline integration, and automated deployment via Vercel.

This report documents the entire frontend architecture, design decisions, development methodology, and deployment strategy of the CareerConnect project, serving as a comprehensive guide to its technical and functional capabilities.

4. Introduction

4.1 Purpose of the Project

The purpose of CareerConnect is to build a scalable and user-friendly frontend platform that acts as a bridge between talented professionals and hiring companies. The application aims to solve the usability issues prevalent in legacy job boards by offering a clean, modern, and highly interactive interface that enhances user engagement and satisfaction.

4.2 Scope

The current scope of the project is strictly focused on the frontend architecture and user interface design. It encompasses the visual layout, client-side routing, state management, and mock data integration to simulate a fully functioning platform. The scope includes public-facing pages (Home, Job Listings, Job Details) and private authenticated views (Candidate Dashboard, Recruiter Dashboard). Backend integration and persistent database storage fall outside the current scope and are earmarked for future iterations.

4.3 Objectives

The core objectives of the CareerConnect project are as follows:

- To design a premium, accessible, and responsive user interface using Tailwind CSS and React.
- To implement seamless client-side navigation using React Router.

- To create distinct, feature-rich dashboards tailored to the specific needs of job seekers and recruiters.
- To establish a robust CI/CD pipeline using GitHub Actions to automate the build and deployment process.
- To successfully host the application on a modern cloud platform (Vercel) for global accessibility.

5. Problem Statement

Despite the abundance of job portals available today, many suffer from outdated user interfaces, poor mobile responsiveness, and cluttered layouts that overwhelm users. Job seekers often find it difficult to filter through irrelevant listings, while recruiters struggle with unintuitive applicant tracking interfaces.

Furthermore, many existing platforms do not cater adequately to modern web standards, resulting in slow page load times and inaccessible design choices. There is a pressing need for a job board that prioritizes user experience (UX), utilizing modern JavaScript frameworks to provide a single-page application (SPA) feel that is fast, engaging, and highly organized.

6. Proposed Solution

To address the aforementioned challenges, CareerConnect proposes a modern Single Page Application (SPA) built with React and Vite. By separating the frontend concerns from the backend logic, the application can deliver lightning-fast navigation and rendering.

The solution includes a meticulously designed UI utilizing Tailwind CSS, ensuring that the platform is 100% responsive across all devices. For job seekers, the platform offers intuitive search and filtering mechanisms, clear job descriptions, and a centralized dashboard to track application statuses. For recruiters, the solution provides an Applicant Tracking System (ATS) styled dashboard with analytics, allowing them to monitor active jobs and manage incoming applications effortlessly.

7. System Architecture

7.1 High-Level Architecture

The CareerConnect application follows a Client-Side Rendering (CSR) architecture. The entire application is bundled into static JavaScript, CSS, and HTML files by the Vite bundler. These static assets are then served globally via Vercel's Edge Network (CDN). When a user navigates to the URL, their browser downloads the initial HTML bundle and subsequently fetches JavaScript to dynamically render the content.

7.2 Component Interaction

The application is built using a component-based architecture in React. The UI is broken down into reusable elements:

- **Layout Components:** MainLayout, Navbar, and Footer wrap the page contents.
- **Page Components:** Home, Jobs, JobDetails, Login, Register, CandidateDashboard, and RecruiterDashboard manage the specific view logic.
- **UI Primitives:** Reusable components like Button and Input handle basic user interactions consistently across the app.

7.3 Data Flow Explanation

Currently, as a frontend-only project, the data flow is strictly client-side. When a user interacts with the UI (e.g., clicking "Apply" or filtering jobs),

the React components update their internal state utilizing React Hooks (`useState`, `useEffect`). Mock data objects (arrays of job listings and candidate profiles) are stored directly within the component state and passed down to child components via props. In a production backend scenario, this flow would be replaced by asynchronous HTTP requests utilizing libraries like Axios to fetch real-time data from RESTful APIs.

8. Technology Stack

The following technologies were carefully selected to build a scalable, maintainable, and modern frontend application:

Technology	Category	Purpose
React (v19)	Frontend Framework	Component-based UI rendering.
TypeScript	Language	Static typing for robust code and better developer experience.
Vite	Build Tool	Lightning-fast development server and optimized production build.
Tailwind CSS	Styling	Utility-first CSS framework for rapid UI development.
React Router	Routing	Client-side routing for seamless Single Page Application transitions.
Lucide React	Assets	Modern, consistent SVG icon library.
Vercel	Hosting	Production deployment platform and CDN edge delivery.
GitHub Actions	CI/CD	Automated pipeline for building and deploying on code push.

8.2 Backend, Database, and APIs

Note: This project is independently focused on the Frontend. Backend architecture, database management, and API integrations are slated for future development phases.

8.3 AI Tools Used During Development

Advanced AI coding assistants (Google DeepMind's Antigravity system) were heavily leveraged to accelerate development. The AI assisted in scaffolding the Vite project, generating Tailwind UI components, configuring TypeScript path aliases, and writing automated CI/CD pipeline scripts.

9. Functional Features

9.1 Home Page

The landing page serves as the entry point for the application. It features a compelling Hero section with a dynamic search bar, allowing users to immediately query jobs by title, skill, or location. Below the hero section, the page displays statistics to build trust, a grid of "Featured Jobs" to draw immediate interest, and a "Top Companies" section showcasing potential employers.

9.2 Job Listing Page (/jobs)

This page displays a comprehensive list of available jobs. It features a two-column layout on desktop: a left sidebar containing various filters (Job Type, Work Mode, Experience Level) and a right main content area displaying job cards. Each job card highlights the company logo, job title, location, salary range, and tags for required skills.

9.3 Search and Filter Jobs

The application implements an intuitive search and filter mechanism. Users can toggle checkboxes in the sidebar to refine their job hunt. The interface updates seamlessly, providing immediate visual feedback to the user without requiring page reloads.

9.4 Job Details Page (/jobs/:id)

When a specific job is selected, this page provides an in-depth view of the role. It includes a comprehensive job description, key responsibilities, requirements, and an overview of the hiring company. A prominent, sticky "Apply Now" button ensures a high conversion rate for applications.

9.5 Responsive UI

The entire application is built with a mobile-first philosophy. Using Tailwind CSS's responsive utility classes, layouts adapt gracefully to different screen sizes. For instance, the filter sidebar on the Jobs page collapses into a modal/dropdown view on mobile devices to preserve screen real estate.

9.6 Authentication (Mocked)

The application includes fully designed Login and Registration pages. The Registration page features a toggle switch allowing users to sign up specifically as a "Job Seeker" or a "Recruiter", dynamically adjusting the form fields (e.g., prompting for a Company Name if 'Recruiter' is selected).

9.7 Admin/Role Features (Dashboards)

Candidate Dashboard: Provides job seekers with a profile completion meter, statistics on jobs they have applied for, saved jobs, and a list of recommended roles based on their search history.

Recruiter Dashboard: Provides employers with an Applicant Tracking System (ATS) interface. It displays high-level metrics (Total Applications, Shortlisted, Hired) and features a data table listing recent applicants with quick-action buttons to approve or reject candidates.

10. User Experience (UX)

Design Decisions

Significant effort was invested in ensuring the application feels "premium" and highly professional.

10.1 Navigation and Layout

The application utilizes a persistent top navigation bar and a fat footer. This standard layout ensures users always know how to traverse the site. The use of React Router enables instantaneous transitions between pages, preventing jarring browser reloads.

10.2 Color Scheme and Typography

The design system is built around a modern palette featuring subtle grays, crisp whites, and a vibrant primary color (teal/cyan) for call-to-action elements. This contrast directs the user's attention to interactive elements. The typography relies on highly legible sans-serif fonts (like Inter or Roboto) suitable for long reading sessions typical in job searching.

10.3 Accessibility and Responsiveness

Semantic HTML tags (e.g., `<nav>`, `<main>`, `<aside>`) were used to improve accessibility for screen readers. Buttons and inputs have distinct hover and focus states defined via Tailwind classes, ensuring keyboard navigation is intuitive and visually clear.

11. Development Process

The development of CareerConnect was executed independently, utilizing modern AI tools to maximize productivity and code quality.

11.1 AI-Assisted UI and Code Generation

AI was employed to scaffold boilerplate code and generate complex HTML/Tailwind structures. By providing high-level prompts detailing the desired aesthetic (e.g., "Build a premium Candidate Dashboard with Tailwind"), the AI generated React components that were then refined and integrated manually into the application's routing structure.

11.2 Iterative Bug Fixing

During development, TypeScript enforced strict type checking. When errors occurred (such as unused variables or incorrect component props), AI tools assisted in quickly identifying the root cause and applying targeted, multi-line replacements without disrupting the surrounding codebase.

11.3 CI/CD and Documentation

AI also played a crucial role in DevOps and documentation, specifically in drafting the `ci.yml` GitHub Actions workflow and ensuring Vercel deployment configurations (`vercel.json`) were perfectly optimized for Single Page Applications.

12. GitHub Repository Management

Version control is maintained via Git and hosted on GitHub.

- **Branch Strategy:** The project utilizes a simplified trunk-based development strategy, where all stable feature implementations are committed directly to the main branch.
- **Commit Process:** Commits are made logically (e.g., "Initial commit: CareerConnect frontend", "Add deployment configs") to maintain a readable history.
- **Version Control:** Crucial files like `node_modules` and local environment variables are strictly ignored using a configured `.gitignore` file to keep the repository clean and performant.

13. CI/CD Pipeline

A robust Continuous Integration and Continuous Deployment (CI/CD) pipeline was established using GitHub Actions to automate software delivery.

13.1 Trigger Events

The pipeline is configured to trigger automatically upon any push or pull_request to the main branch. This ensures that no unauthorized or broken code makes it to the production server.

13.2 Build Process

The GitHub Action provisions an Ubuntu runner, installs Node.js, and executes `npm ci` to cleanly install dependencies. It then runs the TypeScript compiler and Vite bundler (`npm run build`).

13.3 Deployment to Vercel

If the build is successful, the workflow utilizes the Vercel CLI. Using secure GitHub Secrets (`VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`), the pipeline pulls the production environment variables and deploys the pre-built static assets directly to Vercel's edge network.

13.4 Benefits of Automation

This automated pipeline completely removes manual deployment errors. The developer can focus entirely on writing frontend code, knowing that pushing to GitHub will instantly update the live website within minutes.

14. Deployment

The application is deployed on **Vercel**, a cloud platform specifically optimized for frontend frameworks. A critical configuration file, `vercel.json`, was added to the repository root. This file includes rewrite rules that redirect all incoming traffic to `index.html`. This is mandatory for React applications using client-side routing; without it, navigating directly to a deep link (e.g., `/candidate/dashboard`) would result in a 404 Not Found error from the server.

15. Testing and Validation

Validation of the project primarily revolves around strict compilation testing and manual UI/UX reviews.

- **Type Checking:** The TypeScript compiler (tsc) acts as the first line of defense, catching type mismatches and undefined variables before the build completes.
- **Linting:** ESLint is configured to enforce code style and catch logical errors specific to React Hooks.
- **Manual QA:** Extensive manual testing was performed across different viewport sizes (Mobile, Tablet, Desktop) using Chrome Developer Tools to ensure the Tailwind classes behaved as expected.

16. Challenges Encountered and Solutions

Challenge: TypeScript Configuration Deprecations.

During the build process, the TypeScript compiler threw errors regarding deprecated properties (`baseUrl` and `ignoreDeprecations`).

Solution: The `tsconfig.app.json` was analyzed and refactored. The deprecated `baseUrl` was removed entirely, as modern TypeScript (v5.0+) resolves paths aliases directly relative to the configuration file. This fixed the build issues cleanly without hacky workarounds.

Challenge: Client-Side Routing on Deployment.

React Router relies on the browser's History API, which can cause 404

errors on hosting platforms when refreshing pages.

Solution: Implemented a `vercel.json` rewrite rule to funnel all requests to `index.html`, ensuring the React application handles the routing internally.

17. Future Enhancements

As an independently developed project currently focused on the frontend, there are several key areas targeted for future enhancement:

- **Backend Integration:** Developing a robust REST API using Spring Boot or Node.js to replace the current client-side mock data.
- **Database Implementation:** Setting up a relational database (e.g., PostgreSQL or MySQL) to persist user profiles, job listings, and application histories.
- **Authentication:** Implementing JWT-based authentication or OAuth (Google/LinkedIn login) for secure user sessions.
- **State Management:** Integrating Redux or Zustand to handle complex global state across the dashboards as the application grows in complexity.

18. Conclusion

The CareerConnect frontend project successfully demonstrates the ability to architect, design, and deploy a complex, modern web application. By leveraging Vite, React, and Tailwind CSS, the project achieves a highly responsive and aesthetically pleasing user interface that solves real-world usability issues found in traditional job boards.

Furthermore, the integration of an automated GitHub Actions CI/CD pipeline leading to Vercel deployment showcases a professional, industry-

standard development workflow. This project stands as a solid foundation, fully prepared for future backend integration and scaling.

19. References

- React Documentation: <https://react.dev/>
- Vite Bundler: <https://vitejs.dev/>
- Tailwind CSS: <https://tailwindcss.com/>
- Vercel Deployment: <https://vercel.com/docs>
- GitHub Actions: <https://docs.github.com/en/actions>

20. Appendix

(Placeholders for Application Screenshots)

- [Screenshot 1: Landing Page Hero Section]
- [Screenshot 2: Job Listing Page with Filter Sidebar]
- [Screenshot 3: Candidate Dashboard Overview]
- [Screenshot 4: Recruiter ATS Table View]
- [Screenshot 5: Responsive Mobile View of Navbar]